PYTHON PROGRAMMING CASE STUDIES

CASE STUDIES 11–20 — COMPLETE ASSIGNMENT

All ten requested headings, with detailed step-by-step flowcharts included inside each case study.

Required headings in every case study

1. Problem Statement · 2. Objectives · 3. Assumptions · 4. Algorithm · 5. Flow Chart · 6. Pseudocode · 7. Python Source Code · 8. Test Cases · 9. Output Screen (sample output) · 10. Conclusion

Flowchart note: Each flowchart uses separate boxes and arrows. Decision boxes are highlighted and the Yes/No outcomes are written explicitly. Follow the flow downward and use the stated return steps for loops.

11. Examination Result and Grace-Mark Validation

1. Problem Statement

Process marks for multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

2. Objectives

• Validate each subject's marks.
• Apply grace marks only when the shortage is within the policy limit.
• Produce subject-wise results, total, percentage, and classification.

3. Assumptions

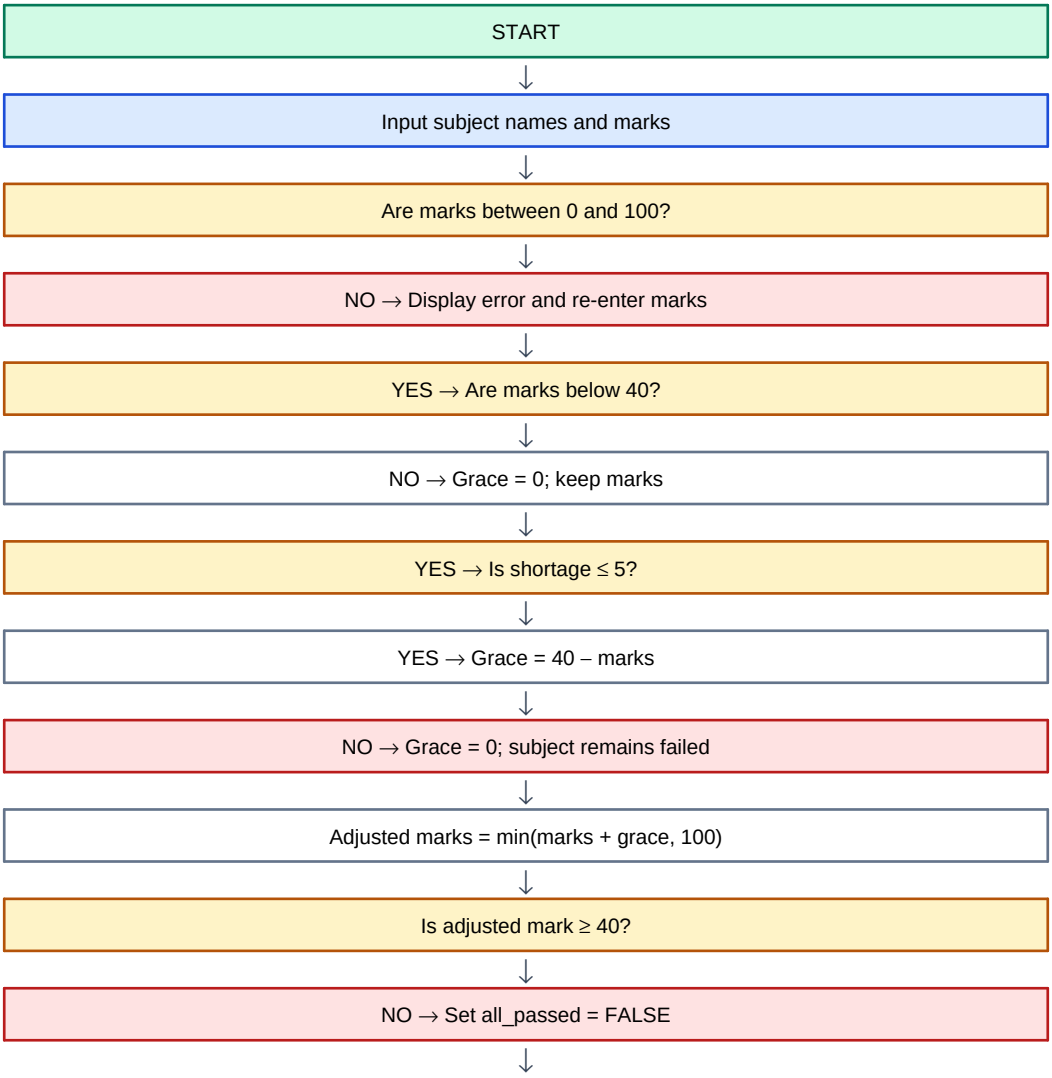
• Each subject is out of 100 and pass mark is 40.
• Up to 5 grace marks may be added only when a student is short by 5 marks or fewer.
• Adjusted marks cannot exceed 100.
• If any subject remains below 40, the final result is FAIL. Otherwise: Distinction ≥75%, First Class ≥60%, Second Class ≥50%, else Pass.

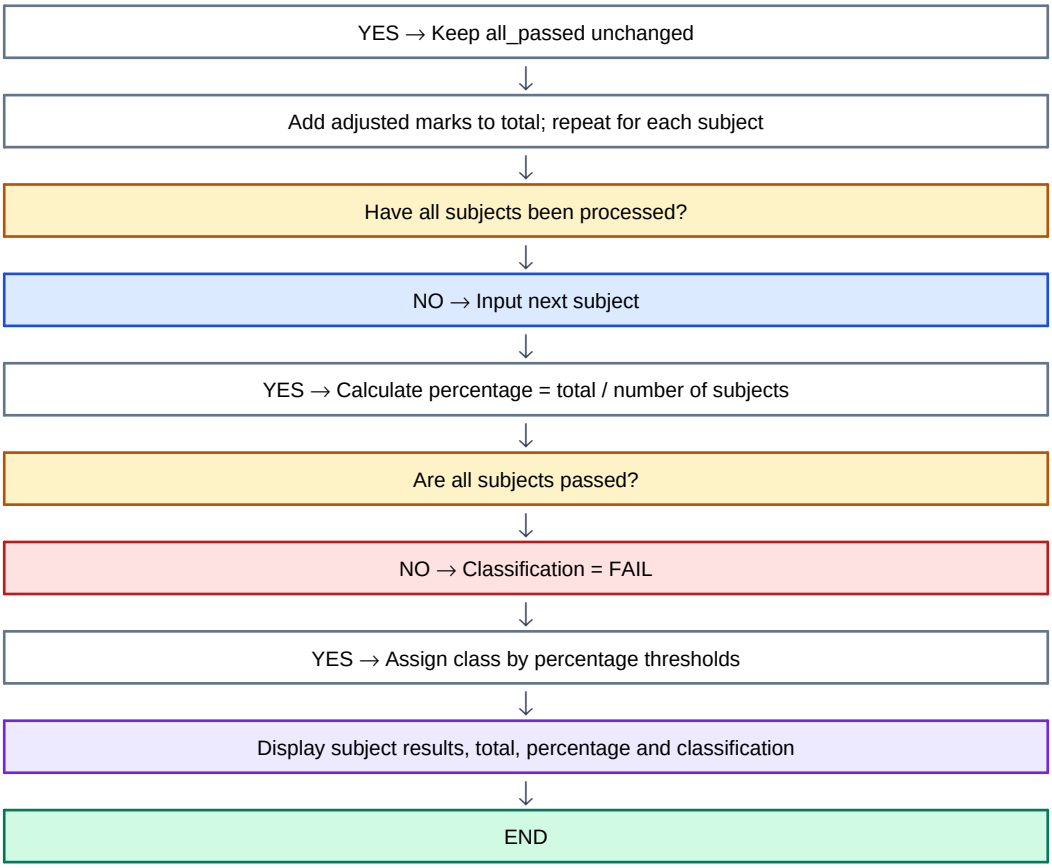
4. Algorithm

1. Start and initialize total marks and pass status.
2. Read each subject's marks and validate that marks are from 0 to 100.
3. If marks are below 40 and shortage is at most 5, add exactly the shortage as grace marks.
4. Keep marks within the maximum of 100.
5. Check whether each adjusted subject mark is at least 40 and add it to the total.
6. After all subjects, calculate percentage and classification.
7. Display subject-wise original marks, grace marks, adjusted marks, total, percentage, and result.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.





6. Pseudocode

```
START
  total ← 0
  all_passed ← TRUE
  FOR each subject
    INPUT subject, marks
    IF marks < 0 OR marks > 100
      DISPLAY "Invalid marks"
    ELSE
      grace ← 0
      IF marks < 40 AND (40 - marks) ≤ 5
        grace ← 40 - marks
      adjusted ← MIN(marks + grace, 100)
      IF adjusted < 40
        all_passed ← FALSE
      total ← total + adjusted
      DISPLAY subject, marks, grace, adjusted
  percentage ← total / number_of_subjects
  IF all_passed = FALSE: result ← "FAIL"
  ELSE IF percentage ≥ 75: result ← "DISTINCTION"
  ELSE IF percentage ≥ 60: result ← "FIRST CLASS"
  ELSE IF percentage ≥ 50: result ← "SECOND CLASS"
  ELSE: result ← "PASS"
  DISPLAY total, percentage, result
END
```

7. Python Source Code

```
subjects = {"Python": 38, "Maths": 67, "English": 82, "Physics": 39}
PASS_MARK, MAX_MARK, MAX_GRACE = 40, 100, 5
results = []
all_passed = True

for subject, marks in subjects.items():
    if not 0 <= marks <= MAX_MARK:
        raise ValueError(f"Invalid marks for {subject}")
    grace = PASS_MARK - marks if 0 < PASS_MARK - marks <= MAX_GRACE else 0
    adjusted = min(marks + grace, MAX_MARK)
    results.append((subject, marks, grace, adjusted))
    if adjusted < PASS_MARK:
        all_passed = False

total = sum(row[3] for row in results)
percentage = total / len(results)
if not all_passed:
    classification = "FAIL"
elif percentage >= 75:
    classification = "DISTINCTION"
elif percentage >= 60:
    classification = "FIRST CLASS"
elif percentage >= 50:
    classification = "SECOND CLASS"
else:
```

```
    classification = "PASS"

print("SUBJECT       MARKS  GRACE  FINAL")
for subject, marks, grace, final in results:
    print(f"{subject:<13} {marks:>5}  {grace:>5}  {final:>5}")
print(f"Total: {total}/{len(results) * MAX_MARK}")
print(f"Percentage: {percentage:.2f}%")
print("Classification:", classification)
```

8. Test Cases

Test input / condition	Expected result
Python = 38	Grace = 2; final = 40
Physics = 39	Grace = 1; final = 40
Subject = 30	No grace; subject remains failed
Marks = 105	Invalid input

9. Output Screen (sample output)

10. Conclusion

The program validates marks, applies the grace policy, prevents adjusted marks from exceeding the maximum, and generates a clear subject-wise result and classification.

12. Attendance Recovery Computation

1. Problem Statement

Calculate current attendance and determine the minimum consecutive classes a student must attend to reach a specified eligibility threshold, while checking different future attendance scenarios.

2. Objectives

• Calculate current attendance percentage.
• Find the minimum consecutive classes needed to reach the target.
• Calculate eligibility under different future attendance scenarios.

3. Assumptions

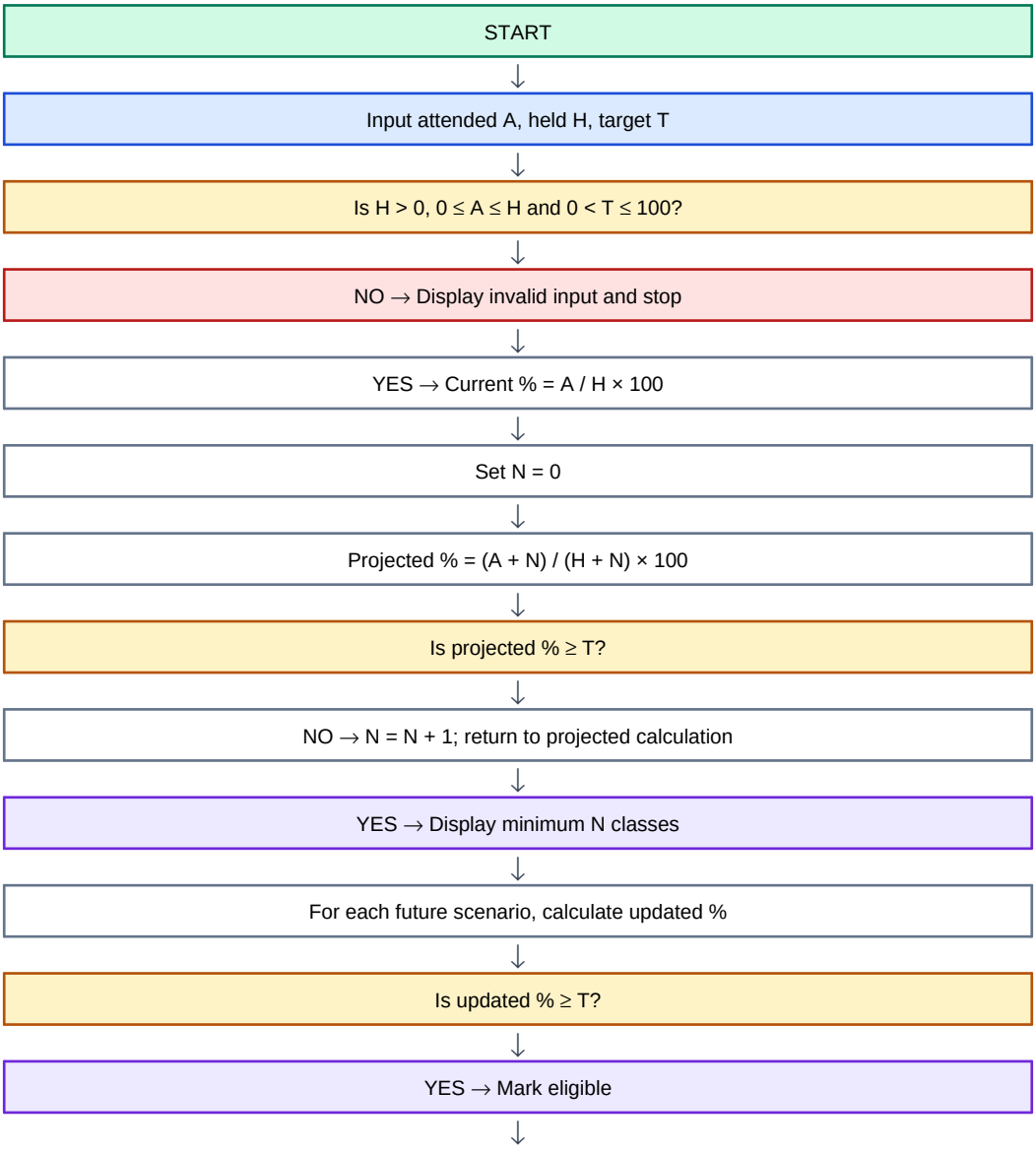
• Attendance = attended classes / held classes × 100.
• Held classes must be greater than zero and attended classes must be between zero and held classes.
• Future recovery classes are all attended.
• Default eligibility threshold is 75%.

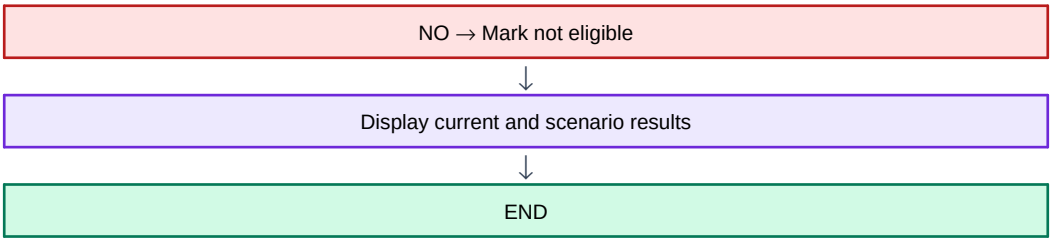
4. Algorithm

1. Input attended classes, held classes, and target percentage.
2. Validate all values.
3. Calculate current attendance percentage.
4. Start additional classes at zero and calculate projected attendance.
5. Increase the additional-class count until projected attendance reaches the target.
6. Calculate scenario percentages and print eligibility.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.





6. Pseudocode

```
START
  INPUT attended A, held H, target T
  IF H ≤ 0 OR A < 0 OR A > H OR T ≤ 0 OR T > 100
    DISPLAY "Invalid input"; STOP
  current ← 100 × A / H
  N ← 0
  WHILE 100 × (A + N) / (H + N) < T
    N ← N + 1
  DISPLAY current, N
  FOR each future scenario (future_attended, future_held)
    updated ← 100 × (A + future_attended) / (H + future_held)
    DISPLAY updated and eligibility
END
```

7. Python Source Code

```
attended, held, target = 42, 60, 75
if held <= 0 or not 0 <= attended <= held or not 0 < target <= 100:
    raise ValueError("Invalid attendance data")

current = attended / held * 100
needed = 0
while (attended + needed) / (held + needed) * 100 < target:
    needed += 1

print(f"Current attendance: {current:.2f}%")
print(f"Consecutive classes required: {needed}")
print(f"After attending: {(attended + needed) / (held + needed) * 100:.2f}%")

scenarios = [(5, 5), (8, 10), (6, 10)]
for future_attended, future_held in scenarios:
    updated = (attended + future_attended) / (held + future_held) * 100
    status = "Eligible" if updated >= target else "Not eligible"
    print(f"Scenario {future_attended}/{future_held}: {updated:.2f}% - {status}")
```

8. Test Cases

Test input / condition	Expected result
42/60 classes	Current 70%; need 12 consecutive classes
45/60 classes	75%; need 0 classes
Attended > held	Invalid input
Held = 0	Invalid input

9. Output Screen (sample output)

10. Conclusion

The program identifies the current attendance level, minimum classes required for recovery, and eligibility under different future scenarios.

13. ATM Transaction Sequence Simulation

1. Problem Statement

Process deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

2. Objectives

• Update balance for successful transactions.
• Reject invalid amounts, withdrawals over the limit, and withdrawals exceeding balance.
• Display successful and rejected transaction counts and closing balance.

3. Assumptions

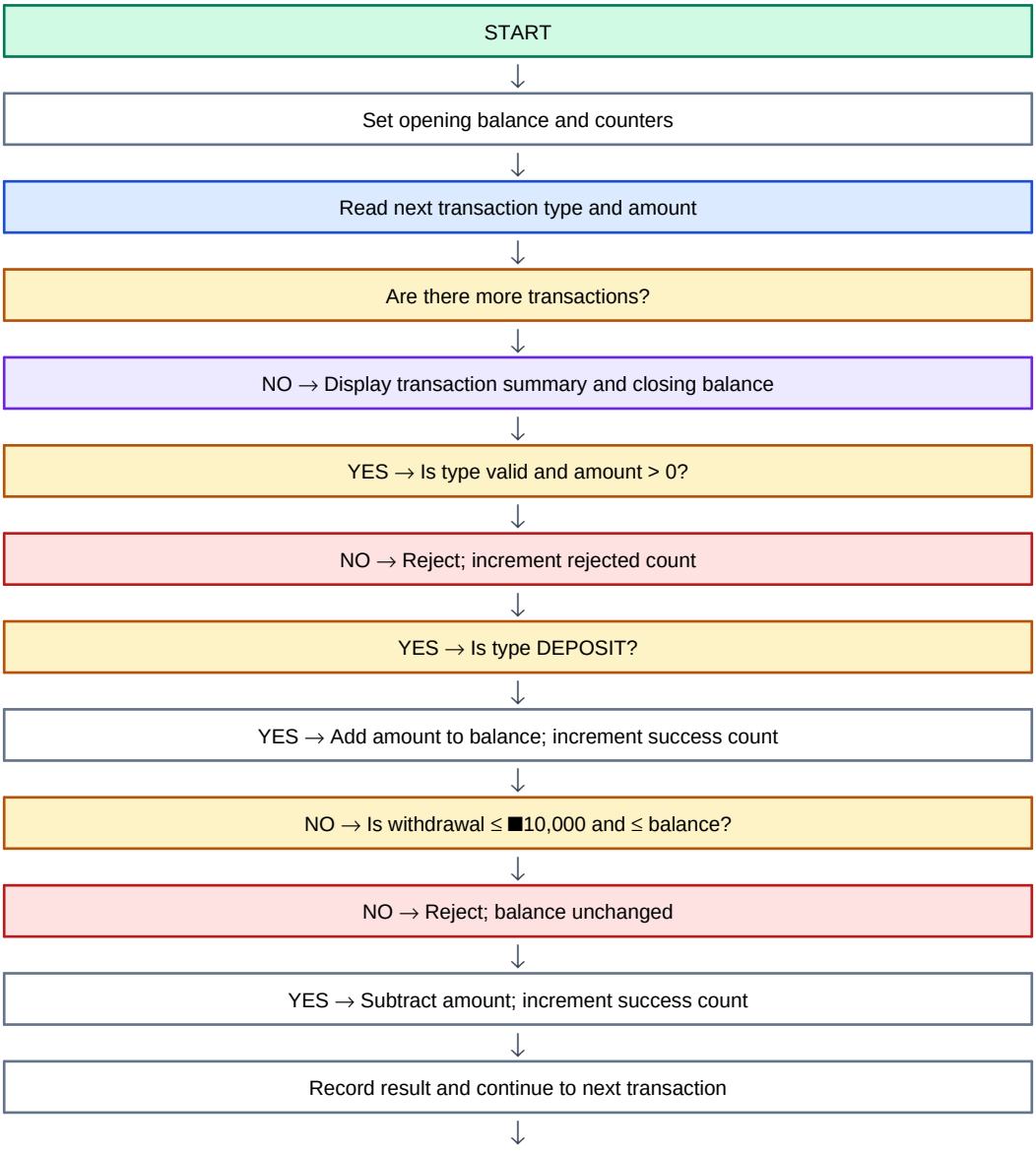
• Opening balance is ■5,000 in the sample.
• Amounts must be positive.
• Maximum withdrawal per transaction is ■10,000.
• Only DEPOSIT and WITHDRAW transaction types are valid.

4. Algorithm

1. Initialize balance and counters.
2. Read each transaction type and amount.
3. Reject invalid type or non-positive amount.
4. For deposits, add amount to balance.
5. For withdrawals, check transaction limit and available balance.
6. Update counters and display closing summary.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.



END after all transactions

6. Pseudocode

```
START
  balance ← opening_balance
  successful ← 0; rejected ← 0
  FOR each transaction
    INPUT type, amount
    IF type invalid OR amount ≤ 0
      reject; rejected ← rejected + 1
    ELSE IF type = DEPOSIT
      balance ← balance + amount
      successful ← successful + 1
    ELSE IF amount > 10000 OR amount > balance
      reject; rejected ← rejected + 1
    ELSE
      balance ← balance - amount
      successful ← successful + 1
  DISPLAY successful, rejected, balance
END
```

7. Python Source Code

```
balance = 5000.00
withdrawal_limit = 10000.00
transactions = [("DEPOSIT", 2000), ("WITHDRAW", 1500),
                ("WITHDRAW", 7000), ("WITHDRAW", 12000),
                ("DEPOSIT", 0)]
successful = rejected = 0

for kind, amount in transactions:
    if kind not in ("DEPOSIT", "WITHDRAW") or amount <= 0:
        print(f"REJECTED: {kind} {amount} (invalid transaction)")
        rejected += 1
    elif kind == "DEPOSIT":
        balance += amount
        print(f"SUCCESS: Deposit {amount:.2f}; balance {balance:.2f}")
        successful += 1
    elif amount > withdrawal_limit:
        print(f"REJECTED: Withdrawal {amount:.2f} exceeds limit")
        rejected += 1
    elif amount > balance:
        print(f"REJECTED: Withdrawal {amount:.2f}; insufficient funds")
        rejected += 1
    else:
        balance -= amount
        print(f"SUCCESS: Withdrawal {amount:.2f}; balance {balance:.2f}")
        successful += 1

print("Successful transactions:", successful)
print("Rejected transactions:", rejected)
print(f"Closing balance: {balance:.2f}")
```

8. Test Cases

Test input / condition	Expected result
Deposit 2000	Balance increases by 2000
Withdraw 7000 when balance 5500	Rejected; balance unchanged
Withdraw 12000	Rejected by limit
Amount 0	Rejected

9. Output Screen (sample output)

10. Conclusion

The simulation demonstrates safe transaction validation, balance updates, and a clear summary without accessing a real bank account.

14. Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, incoming quantities, dispatches, and recorded closing stock for multiple products; detect discrepancies and calculate replenishment requirements.

2. Objectives

• Calculate expected closing stock.
• Detect differences between expected and recorded stock.
• Calculate the quantity needed to reach target stock.

3. Assumptions

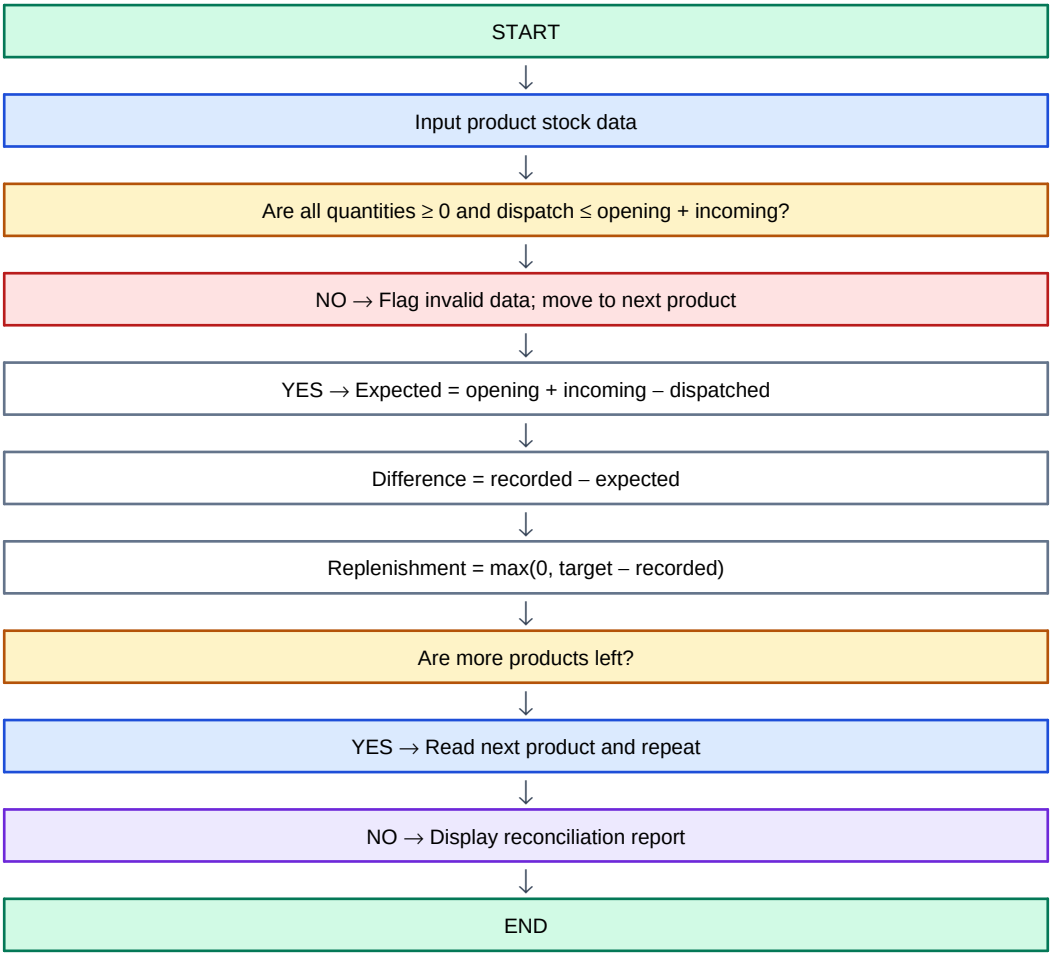
• All quantities must be non-negative whole numbers.
• Expected closing = opening + incoming – dispatched.
• Dispatch cannot exceed opening plus incoming stock.
• Difference = recorded closing – expected closing.

4. Algorithm

1. Input stock details for each product.
2. Validate quantities and available stock.
3. Calculate expected closing stock.
4. Calculate discrepancy and replenishment requirement.
5. Repeat for all products and display report.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.



6. Pseudocode

```
START
  FOR each product
    INPUT opening, incoming, dispatched, recorded, target
    IF any quantity < 0 OR dispatched > opening + incoming
      DISPLAY "Invalid stock data"
    ELSE
      expected ← opening + incoming - dispatched
      difference ← recorded - expected
      replenish ← MAX(0, target - recorded)
```

```
      DISPLAY expected, difference, replenish
  END FOR
END
```

7. Python Source Code

```
inventory = [
    {"name": "Rice", "opening": 100, "incoming": 40, "dispatched": 70, "recorded": 68, "target": 100},
    {"name": "Wheat", "opening": 80, "incoming": 20, "dispatched": 35, "recorded": 65, "target": 90},
    {"name": "Oil", "opening": 50, "incoming": 10, "dispatched": 25, "recorded": 35, "target": 50},
]
print("PRODUCT  EXPECTED  RECORDED  DIFFERENCE  REPLENISH")
for p in inventory:
    vals = [p["opening"], p["incoming"], p["dispatched"], p["recorded"], p["target"]]
    if any(v < 0 for v in vals) or p["dispatched"] > p["opening"] + p["incoming"]:
        print(p["name"], "INVALID DATA")
        continue
    expected = p["opening"] + p["incoming"] - p["dispatched"]
    difference = p["recorded"] - expected
    replenish = max(0, p["target"] - p["recorded"])
    print(f"{p['name']:<8} {expected:>8} {p['recorded']:>9} {difference:>11} {replenish:>10}")
```

8. Test Cases

Test input / condition	Expected result
Opening 100, incoming 20, dispatched 30	Expected closing = 90
Expected 90, recorded 85	Difference = –5
Recorded below target	Replenishment = target – recorded
Dispatch exceeds available stock	Invalid data

9. Output Screen (sample output)

10. Conclusion

The program highlights inventory discrepancies and replenishment needs, helping prevent stock shortages and investigate mismatches.

15. Smart Parking Fee Management

1. Problem Statement

Calculate parking charges from entry and exit times, vehicle categories, daily limits, and penalty conditions while validating time inputs.

2. Objectives

• Validate vehicle category and time format.
• Calculate duration and charge partial hours correctly.
• Apply fee caps and lost-ticket penalty.

3. Assumptions

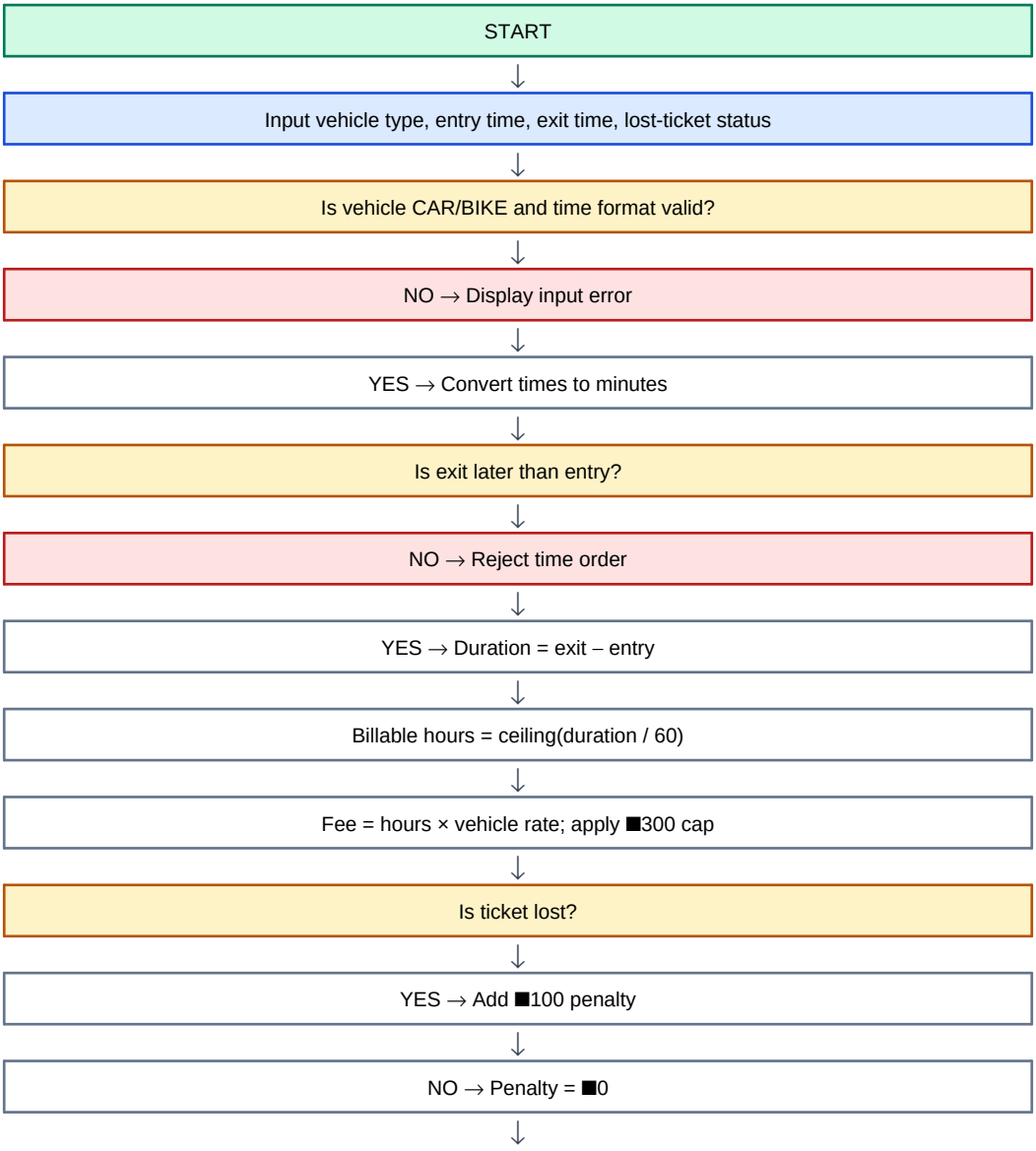
• Times use 24-hour HH:MM format and refer to the same day in this example.
• CAR rate = ■40 per started hour; BIKE rate = ■20 per started hour.
• Base fee cap is ■300; lost-ticket penalty is ■100.
• Cross-midnight visits require date-aware input and are outside this simplified program.

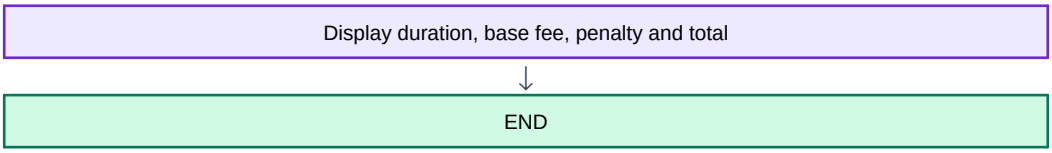
4. Algorithm

1. Input vehicle type, entry and exit times, and lost-ticket status.
2. Validate category and time format.
3. Convert times to minutes and check exit is later than entry.
4. Round duration up to the next hour.
5. Calculate rate-based fee and apply cap.
6. Add penalty if ticket is lost; display bill.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.





6. Pseudocode

```
START
  INPUT vehicle, entry, exit, lost_ticket
  IF vehicle not CAR or BIKE: display error; STOP
  Convert entry and exit to minutes
  IF exit ≤ entry: display invalid time order; STOP
  duration ← exit - entry
  hours ← CEILING(duration / 60)
  rate ← 40 if CAR else 20
  fee ← MIN(hours × rate, 300)
  IF lost_ticket: fee ← fee + 100
  DISPLAY duration, fee
END
```

7. Python Source Code

```
from datetime import datetime
from math import ceil

vehicle = "CAR"
entry_text, exit_text = "09:20", "12:05"
lost_ticket = False
rates = {"CAR": 40, "BIKE": 20}

if vehicle not in rates:
    raise ValueError("Vehicle must be CAR or BIKE")
try:
    entry = datetime.strptime(entry_text, "%H:%M")
    exit_time = datetime.strptime(exit_text, "%H:%M")
except ValueError:
    raise ValueError("Use HH:MM time format")

start = entry.hour * 60 + entry.minute
end = exit_time.hour * 60 + exit_time.minute
if end <= start:
    raise ValueError("Exit must be later than entry on the same day")

duration = end - start
hours = ceil(duration / 60)
base_fee = min(hours * rates[vehicle], 300)
penalty = 100 if lost_ticket else 0
print("Vehicle:", vehicle)
print(f"Duration: {duration // 60} hour(s) {duration % 60} minute(s)")
print("Billable hours:", hours)
print("Base fee: Rs.", base_fee, sep="")
print("Penalty: Rs.", penalty, sep="")
print("Total fee: Rs.", base_fee + penalty, sep="")
```

8. Test Cases

Test input / condition	Expected result
CAR 09:00–10:00	1 hour; fee ■40
BIKE 09:00–10:01	2 hours; fee ■40
Time 27:30	Rejected
Lost ticket = True	Add ■100 penalty

9. Output Screen (sample output)

10. Conclusion

The program validates same-day times and calculates charges using vehicle rates, rounded-up hours, a cap, and an optional penalty.

16. Multi-Day Water Tank Simulation

1. Problem Statement

Simulate tank water levels over several intervals, process inflow and outflow, detect overflow and shortage, and identify the first interval reaching a target level.

2. Objectives

• Update water level for each interval.
• Record overflow and shortage amounts.
• Find the first interval at which the target level is reached.

3. Assumptions

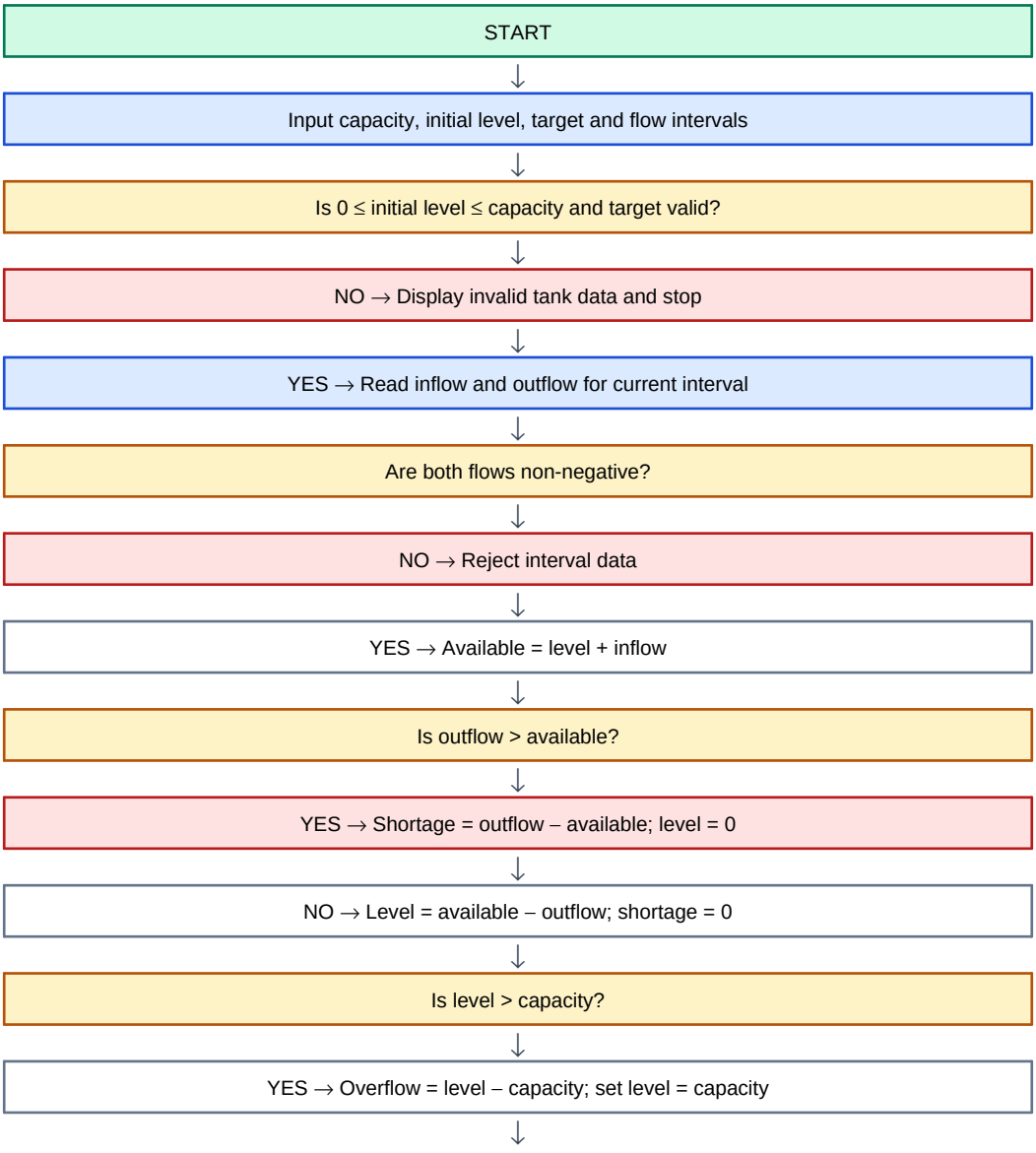
• Initial level must be between zero and tank capacity.
• Inflow and outflow are non-negative.
• Overflow is water above capacity; shortage is outflow beyond available water.
• Actual tank level stays between zero and capacity.

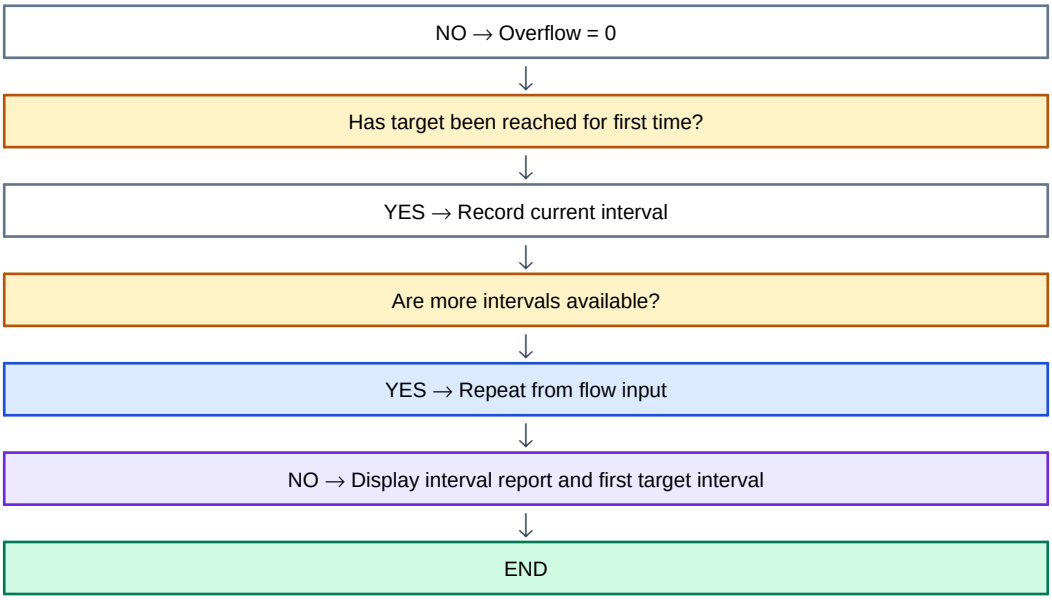
4. Algorithm

1. Input tank capacity, initial level, target, and interval flows.
2. Validate capacity, initial level, target, and flow values.
3. Add inflow to current level.
4. If outflow exceeds available water, record shortage and set level to zero; otherwise subtract outflow.
5. If level exceeds capacity, record overflow and cap it.
6. Record the first interval reaching target; display all intervals.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.





6. Pseudocode

```
START
  INPUT capacity, level, target, intervals
  Validate 0 ≤ level ≤ capacity
  first_target ← NONE
  FOR each interval i
    INPUT inflow, outflow
    IF inflow < 0 OR outflow < 0: report invalid interval
    available ← level + inflow
    IF outflow > available
      shortage ← outflow - available
      level ← 0
    ELSE
      shortage ← 0
      level ← available - outflow
    IF level > capacity
      overflow ← level - capacity
      level ← capacity
    ELSE: overflow ← 0
    IF level ≥ target AND first_target = NONE: first_target ← i
    DISPLAY level, overflow, shortage
  DISPLAY first_target
END
```

7. Python Source Code

```
capacity, level, target = 1000, 400, 700
flows = [(200, 100), (500, 50), (100, 300), (0, 900), (800, 100)]
first_target = None
print("DAY  INFLOW  OUTFLOW  LEVEL  OVERFLOW  SHORTAGE")

for day, (inflow, outflow) in enumerate(flows, start=1):
    if inflow < 0 or outflow < 0:
        raise ValueError("Flow cannot be negative")
    available = level + inflow
    shortage = max(0, outflow - available)
    level = max(0, available - outflow)
    overflow = max(0, level - capacity)
    level = min(level, capacity)
    if level >= target and first_target is None:
        first_target = day
    print(f"{day:>3} {inflow:>7} {outflow:>8} {level:>7} {overflow:>10} {shortage:>9}")

print("First day target reached:", first_target if first_target is not None else "Never")
```

8. Test Cases

Test input / condition	Expected result
Level 900 + inflow 200, capacity 1000	Overflow 100; final level 1000
Level 50, outflow 80, no inflow	Shortage 30; final level 0
Level reaches target	Record first interval
Negative inflow	Reject input

9. Output Screen (sample output)

10. Conclusion

The simulation models changing tank levels, flags overflow and shortages, and reports the first interval when the target is reached.

17. Student Performance Trend Analysis

1. Problem Statement

Compare student scores across examinations, calculate improvement or decline, identify consistently weak subjects, and classify performance.

2. Objectives

• Calculate subject-wise score changes.
• Identify subjects below the weak-subject threshold.
• Classify students using latest average score.

3. Assumptions

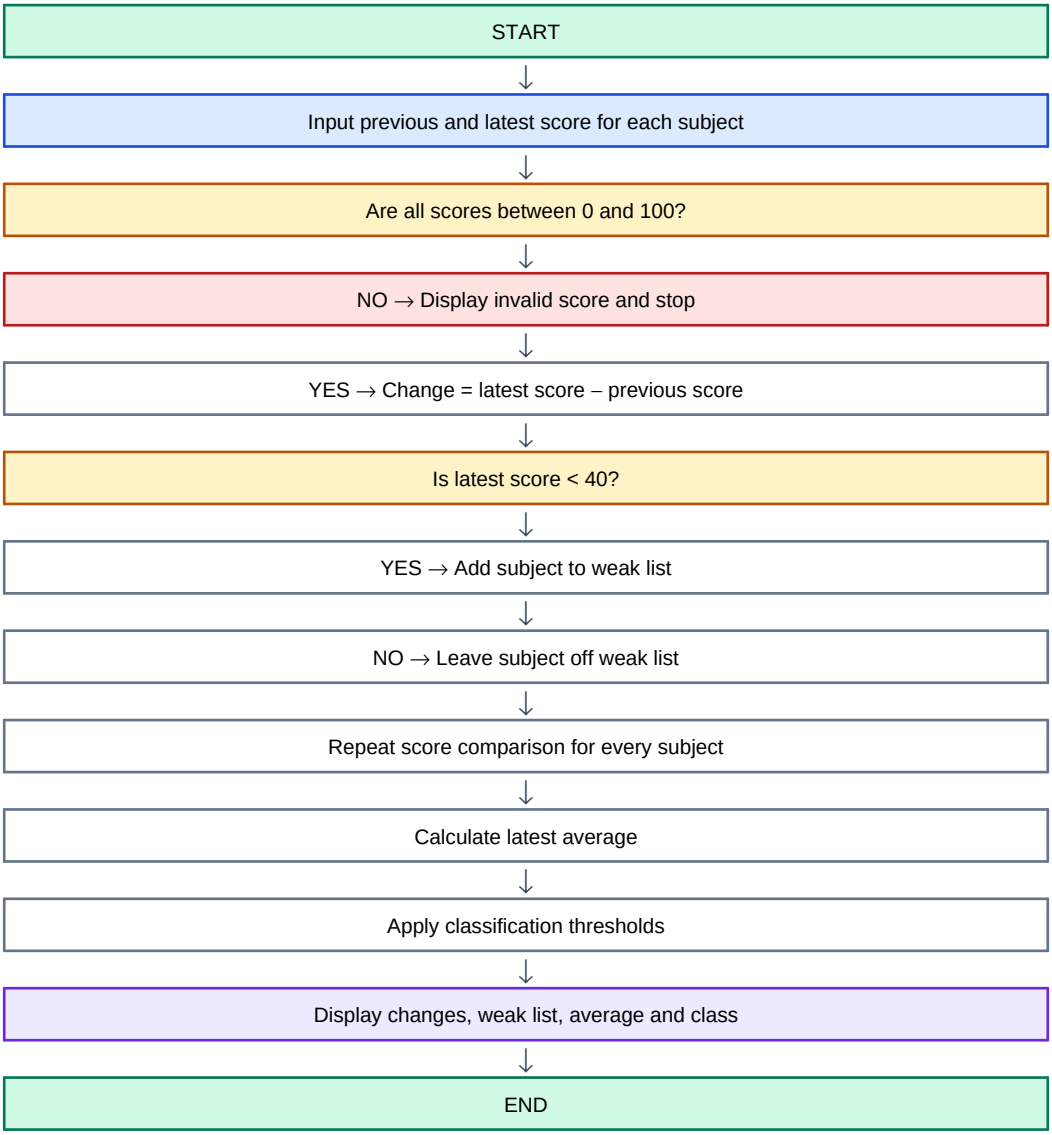
• Scores are percentages between 0 and 100.
• A subject is weak if latest score is below 40.
• Latest average ≥75 = Excellent; ≥60 = Good; ≥40 = Satisfactory; otherwise Needs Improvement.

4. Algorithm

1. Input previous and latest scores for each subject.
2. Validate scores.
3. Calculate latest minus previous score for each subject.
4. Mark latest scores below 40 as weak.
5. Calculate latest average and assign classification.
6. Display score changes and classification.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.



6. Pseudocode

```
START
```

```
  INPUT previous_scores, latest_scores
  Validate every score is between 0 and 100
  weak_subjects ← empty list
  FOR each subject
    change ← latest - previous
    DISPLAY change
    IF latest < 40: add subject to weak_subjects
  average ← sum(latest_scores) / number_of_subjects
  IF average ≥ 75: class ← EXCELLENT
  ELSE IF average ≥ 60: class ← GOOD
  ELSE IF average ≥ 40: class ← SATISFACTORY
  ELSE: class ← NEEDS IMPROVEMENT
  DISPLAY average, weak_subjects, class
END
```

7. Python Source Code

```
previous = {"Python": 55, "Maths": 38, "English": 72, "Physics": 45}
latest = {"Python": 68, "Maths": 35, "English": 78, "Physics": 52}
if previous.keys() != latest.keys():
    raise ValueError("Subjects must match")
if any(not 0 <= x <= 100 for x in list(previous.values()) + list(latest.values())):
    raise ValueError("Scores must be between 0 and 100")

weak = []
print("SUBJECT   PREVIOUS  LATEST  CHANGE")
for subject in previous:
    change = latest[subject] - previous[subject]
    print(f"{subject:<9} {previous[subject]:>8} {latest[subject]:>7} {change:>7}")
    if latest[subject] < 40:
        weak.append(subject)

average = sum(latest.values()) / len(latest)
if average >= 75:
    classification = "EXCELLENT"
elif average >= 60:
    classification = "GOOD"
elif average >= 40:
    classification = "SATISFACTORY"
else:
    classification = "NEEDS IMPROVEMENT"
print(f"Latest average: {average:.2f}%")
print("Weak subjects:", ", ".join(weak) if weak else "None")
print("Classification:", classification)
```

8. Test Cases

Test input / condition	Expected result
55 → 68	Improvement +13
38 → 35	Decline –3; weak subject
Average = 75	Excellent
Score = 101	Invalid input

9. Output Screen (sample output)

10. Conclusion

The program identifies subject trends, weak areas, and an overall performance classification to support targeted improvement.

18. Fuel Consumption and Route Comparison

1. Problem Statement

Compare vehicles for a journey using distance, mileage, fuel price, and reserve requirements, then select the lowest-cost feasible option.

2. Objectives

• Estimate fuel needed and journey cost.
• Include a reserve fuel requirement.
• Select the lowest-cost option with enough available fuel capacity.

3. Assumptions

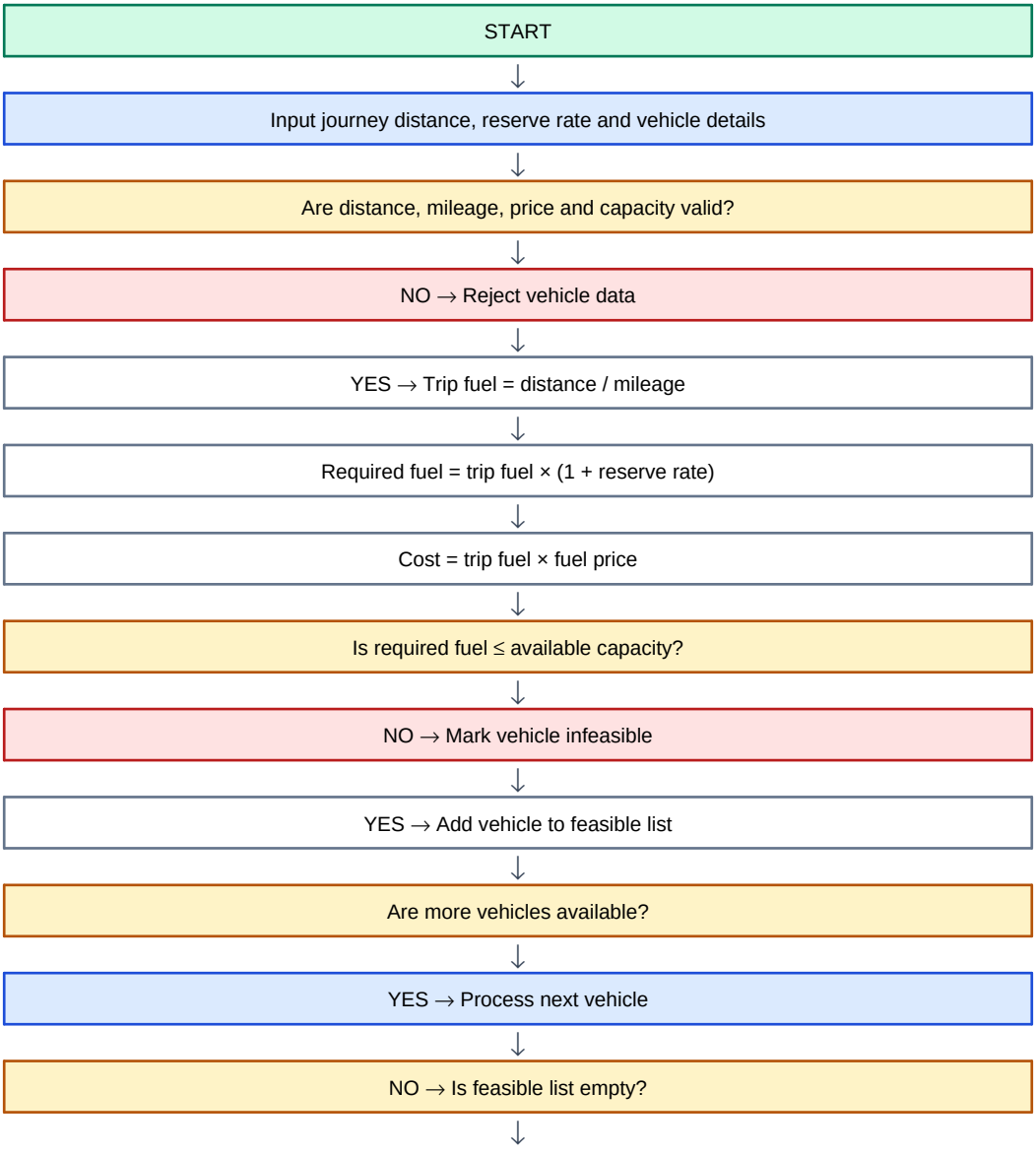
• Distance is in kilometres, mileage in km/litre, and price in rupees per litre.
• Required fuel includes 10% reserve.
• A vehicle is feasible if required fuel does not exceed its available fuel capacity.

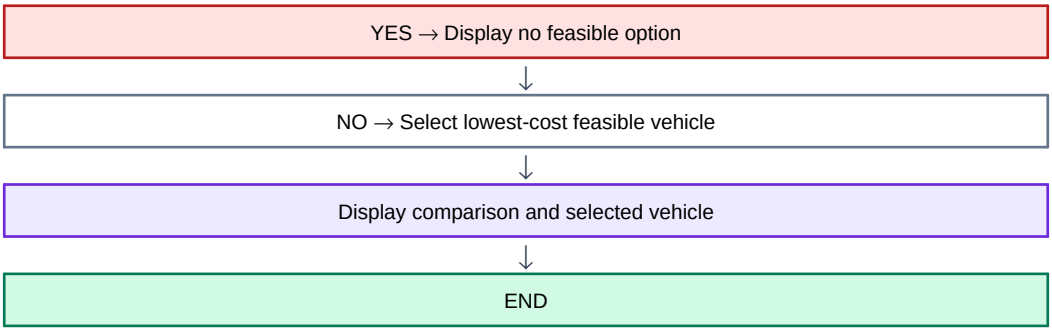
4. Algorithm

1. Input route distance and vehicle details.
2. Validate distance, mileage, price, and capacity.
3. Calculate trip fuel and fuel including reserve.
4. Calculate trip cost and test feasibility.
5. Compare feasible vehicles and select lowest cost.
6. Display comparison and chosen vehicle.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.





6. Pseudocode

```
START
  INPUT distance, reserve_rate, vehicles
  best ← NONE
  FOR each vehicle
    Validate mileage > 0, price > 0, capacity ≥ 0
    trip_fuel ← distance / mileage
    required_fuel ← trip_fuel × (1 + reserve_rate)
    cost ← trip_fuel × price
    feasible ← required_fuel ≤ capacity
    DISPLAY vehicle, fuel, cost, feasibility
    IF feasible AND (best = NONE OR cost < best.cost)
      best ← vehicle
  IF best = NONE: DISPLAY "No feasible vehicle"
  ELSE: DISPLAY best
END
```

7. Python Source Code

```
distance_km, reserve_rate = 300, 0.10
vehicles = [
    {"name": "Car A", "mileage": 15, "price": 105, "capacity": 25},
    {"name": "Car B", "mileage": 20, "price": 105, "capacity": 20},
    {"name": "Car C", "mileage": 12, "price": 100, "capacity": 30},
]
results = []
print("VEHICLE  TRIP_L  WITH_RESERVE  COST(Rs)  FEASIBLE")
for v in vehicles:
    if v["mileage"] <= 0 or v["price"] <= 0 or v["capacity"] < 0:
        print(v["name"], "INVALID DATA")
        continue
    trip_fuel = distance_km / v["mileage"]
    required = trip_fuel * (1 + reserve_rate)
    cost = trip_fuel * v["price"]
    feasible = required <= v["capacity"]
    print(f"{v['name']:<8} {trip_fuel:>6.2f} {required:>13.2f} {cost:>9.2f} {str(feasible):>9}")
    if feasible:
        results.append((cost, v["name"]))
if results:
    cost, name = min(results)
    print(f"Lowest-cost feasible option: {name}, trip cost Rs.{cost:.2f}")
else:
    print("No feasible vehicle for this journey")
```

8. Test Cases

Test input / condition	Expected result
300 km, mileage 15 km/l	Trip fuel = 20 litres
Mileage = 0	Invalid data
Required fuel exceeds capacity	Infeasible
Multiple feasible vehicles	Select minimum cost

9. Output Screen (sample output)

10. Conclusion

The comparison estimates fuel use and cost, includes a reserve, and chooses the cheapest feasible vehicle.

19. Multi-Product Shopping Budget Constraint

1. Problem Statement

Evaluate a basket of products, apply discounts, and reduce quantities to remain within a fixed budget.

2. Objectives

• Calculate subtotals and conditional discounts.
• Compare basket total with the budget.
• Reduce quantities by a defined rule until the total fits the budget.

3. Assumptions

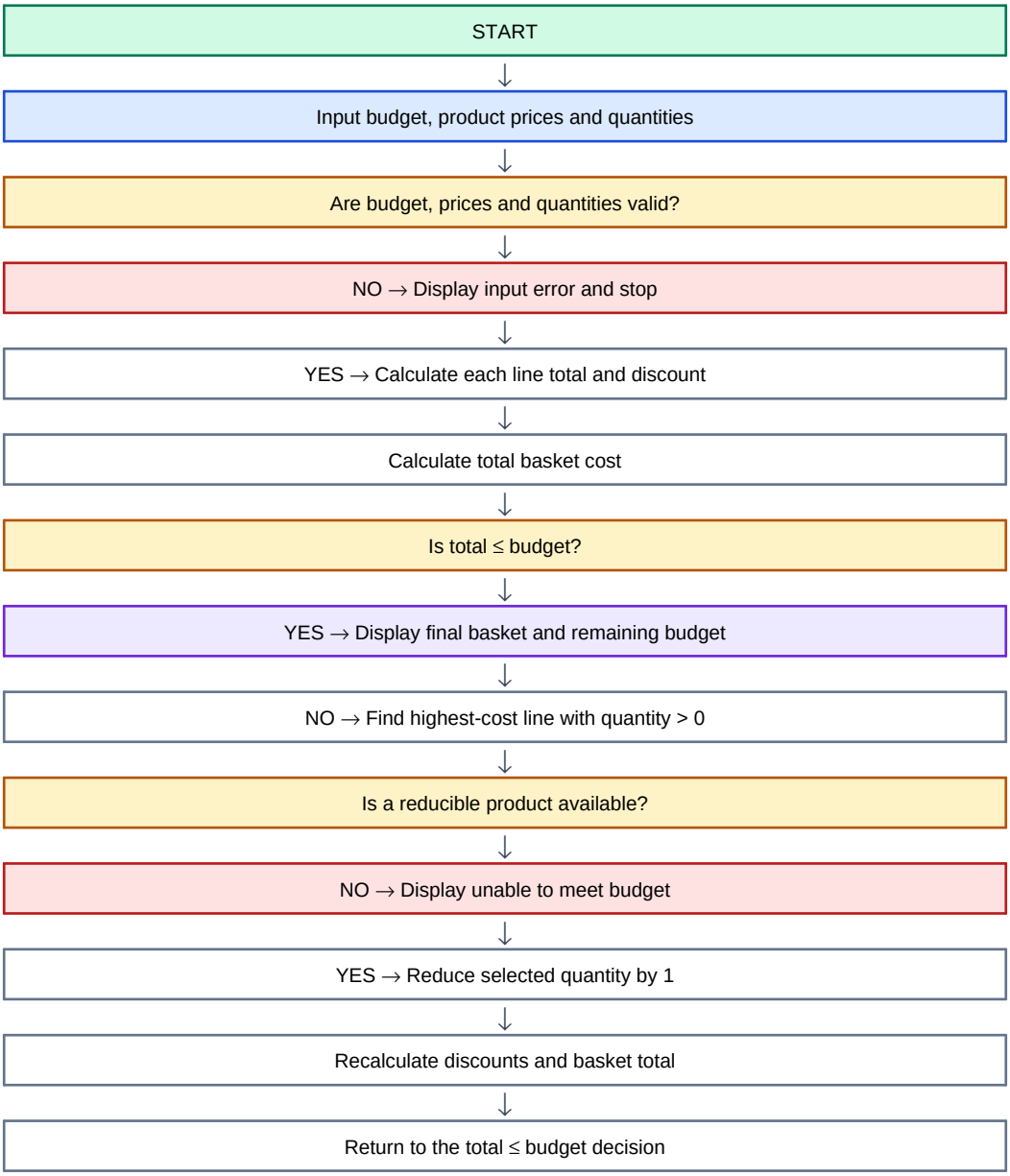
• Prices and quantities are non-negative; quantities are whole numbers.
• A 10% discount applies to a line if quantity is at least 3.
• If over budget, reduce one unit from the product with the largest current line total until within budget.

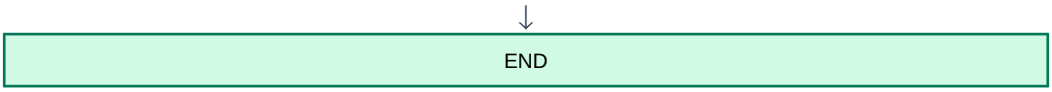
4. Algorithm

1. Input budget and basket.
2. Validate prices and quantities.
3. Calculate each line subtotal and discount.
4. If total exceeds budget, select the largest line total with quantity above zero and reduce quantity by one.
5. Recalculate until within budget or all quantities reach zero.
6. Display original and final total, quantities, and remaining budget.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.





6. Pseudocode

```
START
  INPUT budget, basket
  Validate budget, prices, quantities
  CALCULATE line totals and discounts
  total ← sum(line totals)
  WHILE total > budget AND any quantity > 0
    item ← highest line total with quantity > 0
    item.quantity ← item.quantity - 1
    Recalculate item discount and total
  DISPLAY final basket and total
END
```

7. Python Source Code

```
budget = 1500
basket = [
    {"name": "Notebook", "price": 120, "qty": 4},
    {"name": "Bag", "price": 800, "qty": 1},
    {"name": "Pen pack", "price": 90, "qty": 5},
]
def line_total(item):
    subtotal = item["price"] * item["qty"]
    discount = 0.10 * subtotal if item["qty"] >= 3 else 0
    return subtotal - discount
def total_cost():
    return sum(line_total(item) for item in basket)

if budget < 0 or any(x["price"] < 0 or x["qty"] < 0 for x in basket):
    raise ValueError("Budget, prices and quantities must be non-negative")
original = total_cost()
while total_cost() > budget:
    candidates = [x for x in basket if x["qty"] > 0]
    if not candidates:
        break
    item = max(candidates, key=line_total)
    item["qty"] -= 1

print(f"Original total: Rs.{original:.2f}")
print("FINAL BASKET")
for item in basket:
    print(f"{item['name']}: qty={item['qty']}, line total=Rs.{line_total(item):.2f}")
print(f"Final total: Rs.{total_cost():.2f}")
print(f"Budget remaining: Rs.{budget - total_cost():.2f}")
```

8. Test Cases

Test input / condition	Expected result
Basket under budget	No reductions
Quantity = 3	10% discount applies
Basket over budget	Reduce highest-cost line repeatedly
Negative quantity	Invalid input

9. Output Screen (sample output)

10. Conclusion

The program applies discount rules and reduces purchases using a defined, repeatable policy until the basket fits the budget.

20. University Admission Decision Engine

1. Problem Statement

Evaluate candidates using subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria; generate a ranked list of eligible applicants.

2. Objectives

• Check subject and aggregate thresholds.
• Record reasons for ineligibility.
• Rank eligible candidates consistently using tie-break rules.

3. Assumptions

• Marks are percentages from 0 to 100.
• Engineering eligibility: Mathematics ≥60, Physics ≥50, aggregate ≥70.
• Rank by aggregate descending, then Mathematics descending, then applicant ID ascending.

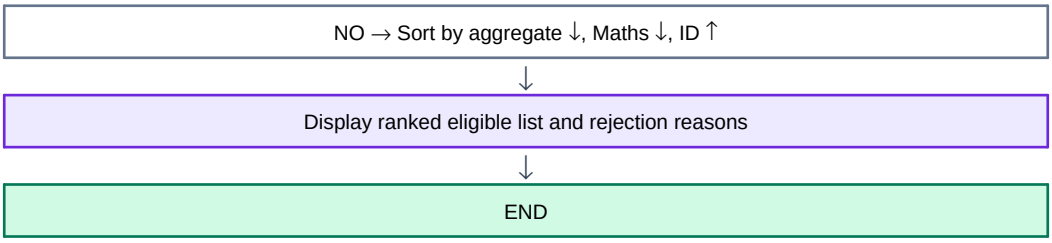
4. Algorithm

1. Input candidate ID, name, aggregate, Mathematics, and Physics marks.
2. Validate all marks.
3. Check Mathematics, Physics, and aggregate thresholds in order.
4. Add eligible candidates to the eligible list; record rejection reasons for others.
5. Sort eligible candidates by the tie-break rules.
6. Display ranked eligible candidates and rejection reasons.

5. Flow Chart — detailed step-by-step

Legend: Green = Start/End; Blue = Input; Yellow = Decision; White = Process; Red = Error/Rejected branch; Purple = Output.





6. Pseudocode

```
START
  INPUT candidate records
  eligible ← empty list; rejected ← empty list
  FOR each candidate
    IF any mark outside 0..100: reject invalid record
    ELSE IF maths < 60: reject with maths reason
    ELSE IF physics < 50: reject with physics reason
    ELSE IF aggregate < 70: reject with aggregate reason
    ELSE: add to eligible
  SORT eligible by aggregate DESC, maths DESC, ID ASC
  DISPLAY ranked eligible list and rejected reasons
END
```

7. Python Source Code

```
candidates = [
    {"id": "A104", "name": "Anu", "aggregate": 88, "maths": 91, "physics": 82},
    {"id": "A102", "name": "Bala", "aggregate": 88, "maths": 94, "physics": 78},
    {"id": "A103", "name": "Charan", "aggregate": 68, "maths": 80, "physics": 70},
    {"id": "A101", "name": "Divya", "aggregate": 79, "maths": 65, "physics": 55},
    {"id": "A105", "name": "Esha", "aggregate": 82, "maths": 70, "physics": 48},
]
eligible, rejected = [], []
for c in candidates:
    marks = [c["aggregate"], c["maths"], c["physics"]]
    if any(not 0 <= m <= 100 for m in marks):
        rejected.append((c, "Invalid marks"))
    elif c["maths"] < 60:
        rejected.append((c, "Mathematics below 60"))
    elif c["physics"] < 50:
        rejected.append((c, "Physics below 50"))
    elif c["aggregate"] < 70:
        rejected.append((c, "Aggregate below 70"))
    else:
        eligible.append(c)

eligible.sort(key=lambda c: (-c["aggregate"], -c["maths"], c["id"]))
print("RANK  ID    NAME    AGGREGATE  MATHS  PHYSICS")
for rank, c in enumerate(eligible, 1):
    print(f"{rank:<5} {c['id']:<5} {c['name']:<7} {c['aggregate']:>9} {c['maths']:>7} {c['physics']:>8}")
print("NOT ELIGIBLE")
for c, reason in rejected:
    print(f"{c['id']} {c['name']}: {reason}")
```

8. Test Cases

Test input / condition	Expected result
Aggregate = 69	Rejected: aggregate below 70
Maths = 59	Rejected: Mathematics below 60
Physics = 49	Rejected: Physics below 50
Equal aggregate	Higher Mathematics ranks first; ID breaks remaining ties

9. Output Screen (sample output)

10. Conclusion

The decision engine applies transparent eligibility rules and tie-breaks to generate a reproducible admissions ranking.